

PENGEMBANGAN APLIKASI MOBILE
State Hoisting, Perubahan Konfigurasi, dan Side-effect
Modul 4



Dosen Pengampu:

Muhammad Aminul Akbar, S.Kom., M.T

Disusun oleh:

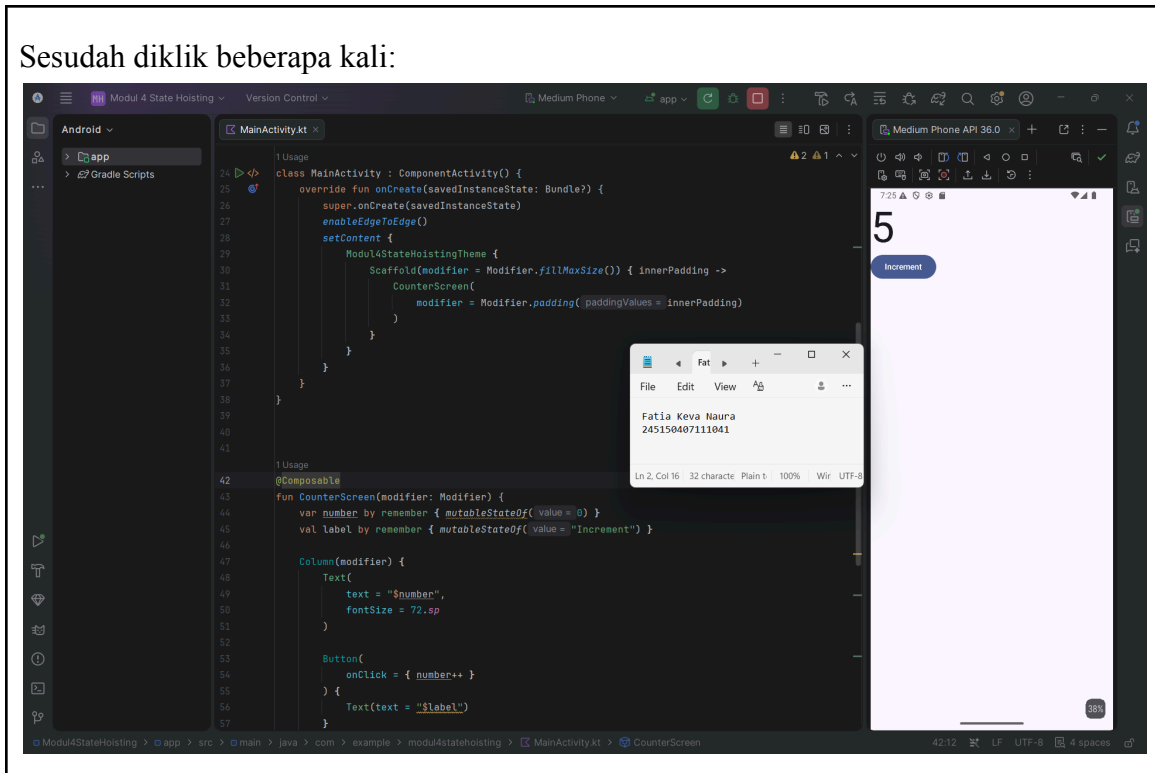
Fatia Keva Naura - 245150407111041

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
2026

4.5.1 State Hoisting

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



- a. Apa yang terjadi setiap kali tombol "Increment" diklik?
= Setiap kali tombol Increment diklik, angka yang ditampilkan akan bertambah 1.
- b. Apa yang menyebabkan hal tersebut terjadi?
= Hal tersebut terjadi karena pada kode tombol terdapat `number++`, sehingga nilai variabel `number` bertambah satu setiap kali tombol ditekan.
- c. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
= State variable yang memengaruhi Text yang menampilkan angka adalah `number`.
- d. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
= State variable yang memengaruhi label Button "Increment" adalah `label`.

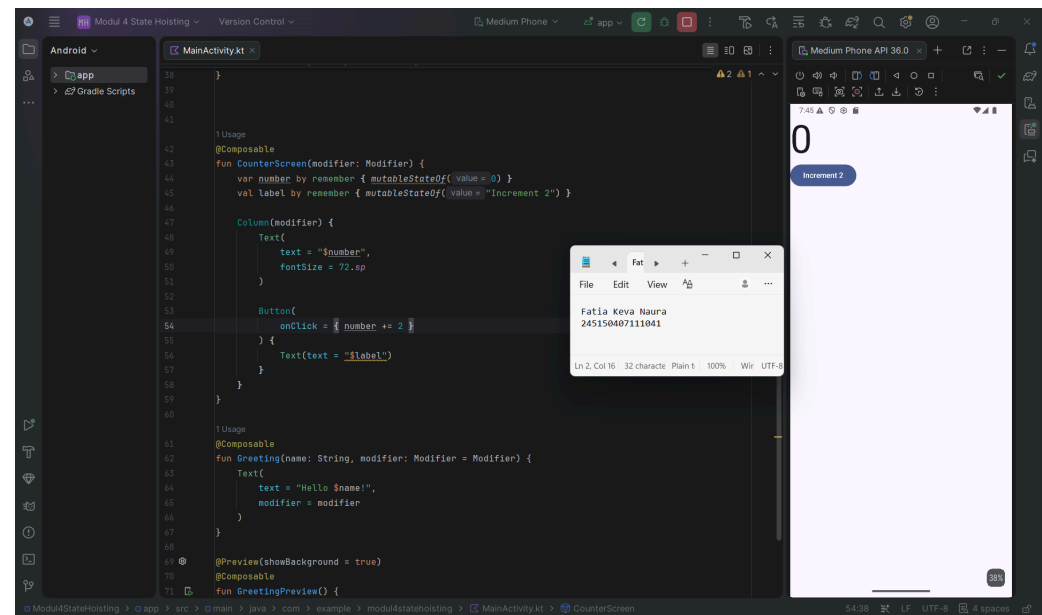
2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable `CounterScreen` agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

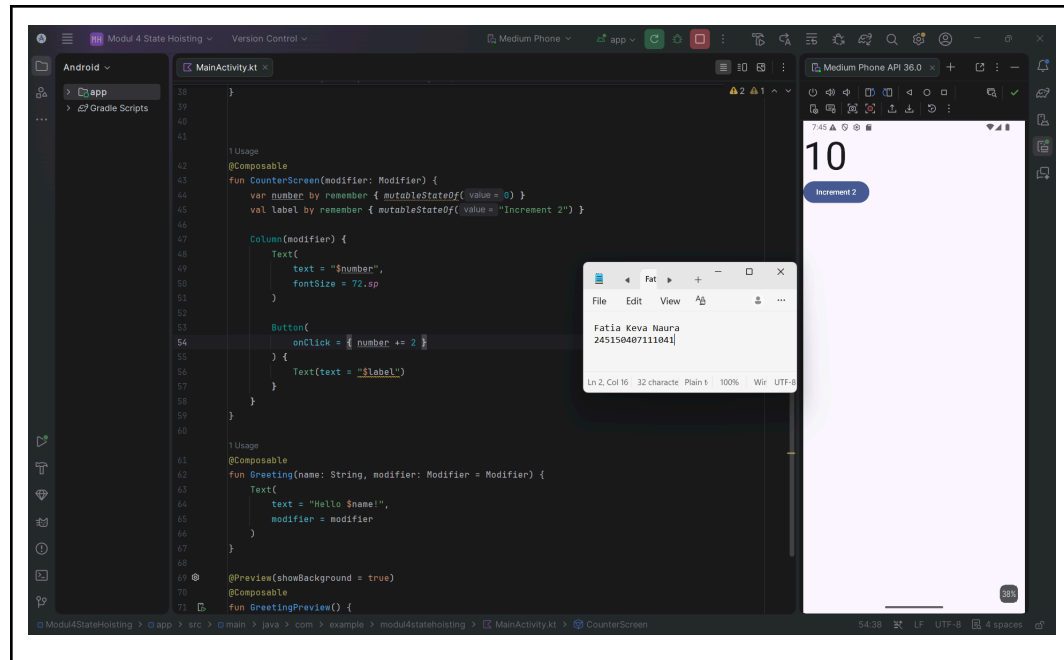
= Perubahan dilakukan pada fungsi `CounterScreen`. Label tombol diubah dari "Increment" menjadi "Increment 2", sedangkan perintah pada `onClick` diubah dari `number++` menjadi `number += 2`. Dengan perubahan tersebut, setiap tombol diklik angka akan bertambah 2.

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.**

Screenshot kondisi awal:

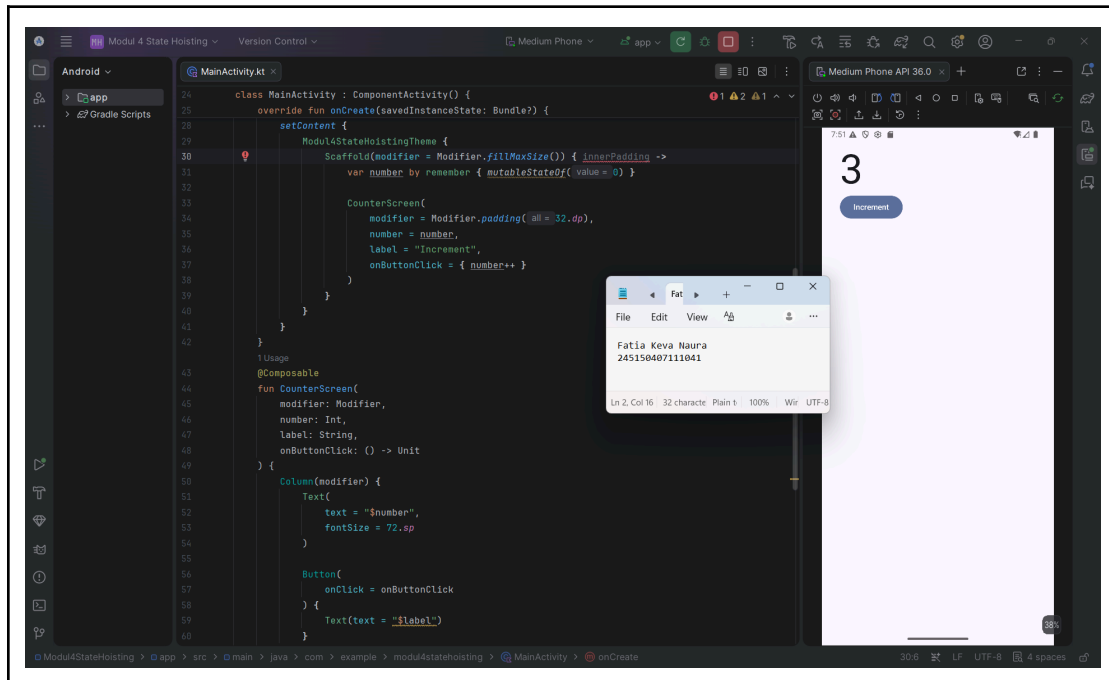


Screenshot setelah tombol diklik beberapa kali:



- Langkah selanjutnya adalah melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable `CounterScreen`.

Jawaban:



4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
- Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?**

= Iya, program dapat berjalan dan tampilannya tetap menampilkan angka dan tombol Increment. Saat tombol diklik, angka bertambah satu.
 - Di bagian kode program manakah saat ini proses increment dilakukan?**

= Proses increment dilakukan di bagian onCreate(), tepatnya pada onClick
= { number++ }.
 - Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?**

= State number dideklarasikan di dalam onCreate(), sedangkan label Button diberikan melalui parameter label = "Increment". Kemudian keduanya dikirim ke CounterScreen sebagai parameter.
5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program:
- Label tombol "Increment" menjadi "Double"**
 - Nilai awal variabel `number` dari 0 menjadi 1.**
 - Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.**

Jawaban a b c:



```
23  
24 // Usage  
25 class MainActivity : AppCompatActivity() {  
26     override fun onCreate(savedInstanceState: Bundle?) {  
27         super.onCreate(savedInstanceState)  
28         enableEdgeToEdge()  
29         setContent {  
30             ModuL4StateHoistingTheme {  
31                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
32                     var number by remember { mutableStateOf( value = 1) }  
33  
34                     CounterScreen(  
35                         modifier = Modifier.padding( all = 32.dp),  
36                         number = number,  
37                         label = "Double",  
38                         onClick = { number *= 2 }  
39                     )  
40                 }  
41             }  
42         }  
43     }  
44 }
```

Preview of the app interface showing a text field with the value "Fatia Keva Nauna 245150407111041" and a button labeled "Double".

- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen setelah perubahan tersebut dilakukan.

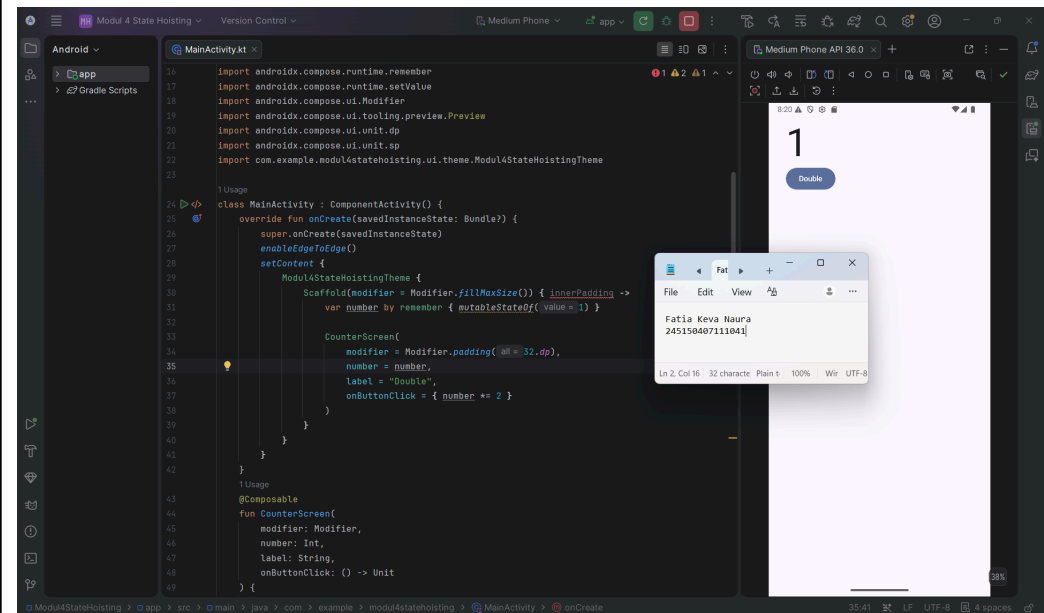
```
= var number by remember { mutableStateOf(1) }
```

```
CounterScreen(  
    modifier = Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```

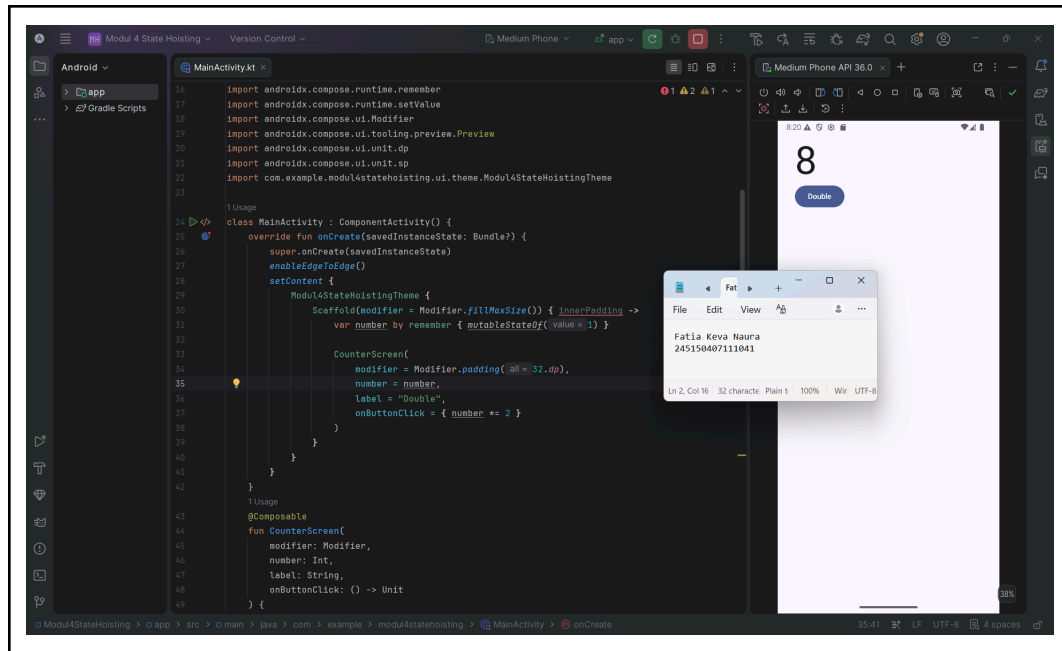
6. Jalankan program,

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Sebelum tombol Double diklik:



Sesudah tombol Double diklik:



b. Di bagian manakah perubahan kode program dilakukan?

= Perubahan kode dilakukan pada fungsi onCreate(), yaitu pada deklarasi state variabel number dan pemanggilan fungsi CounterScreen. Perubahannya meliputi nilai awal number, label tombol, dan fungsi onClick.

c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

= Tidak ada perubahan yang dilakukan di dalam fungsi CounterScreen. Perubahan dilakukan pada bagian onCreate(), sedangkan CounterScreen tetap menerima number, label, dan onClick melalui parameter.

7. Apa arti dari jawaban pada tahap nomor 6 terkait dengan manfaat melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang berkaitan dengan antarmuka pengguna?

= Dari percobaan tersebut dapat dilihat bahwa state hoisting memudahkan ketika terjadi perubahan kebutuhan fungsional pada aplikasi. Perubahan seperti nilai awal, label tombol, dan logika ketika tombol ditekan dapat dilakukan pada bagian parent atau onCreate() tanpa harus mengubah isi CounterScreen. Dengan begitu, CounterScreen menjadi lebih mudah digunakan kembali dan perubahan logika program tidak perlu dilakukan langsung pada komponen antarmukanya.

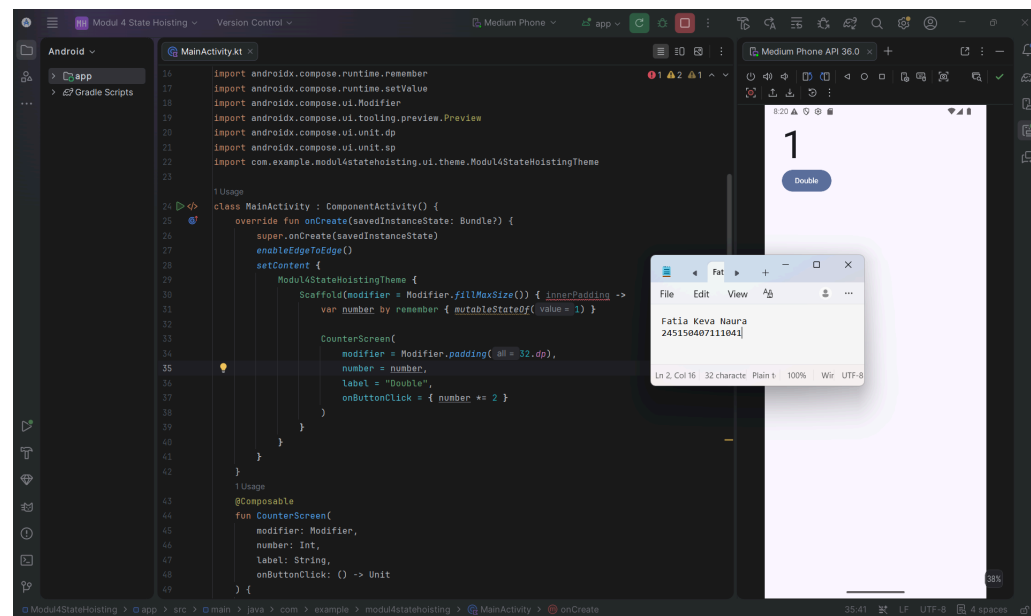
4.5.2 Perubahan Konfigurasi

Aktivitas Percobaan

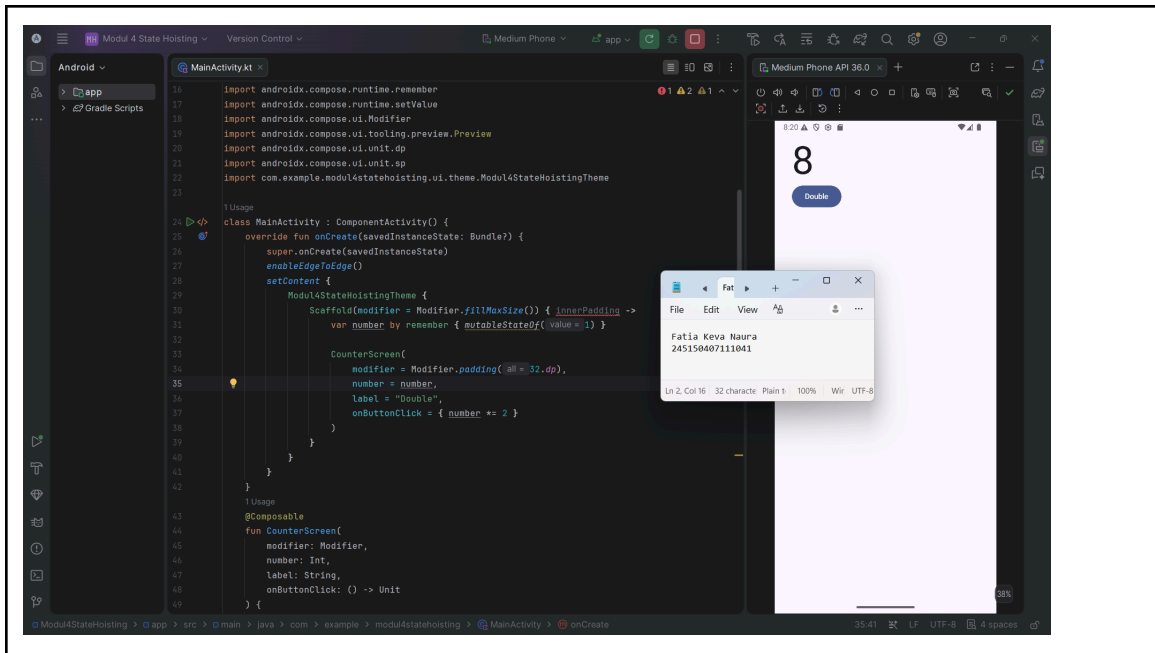
1. Jalankan program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

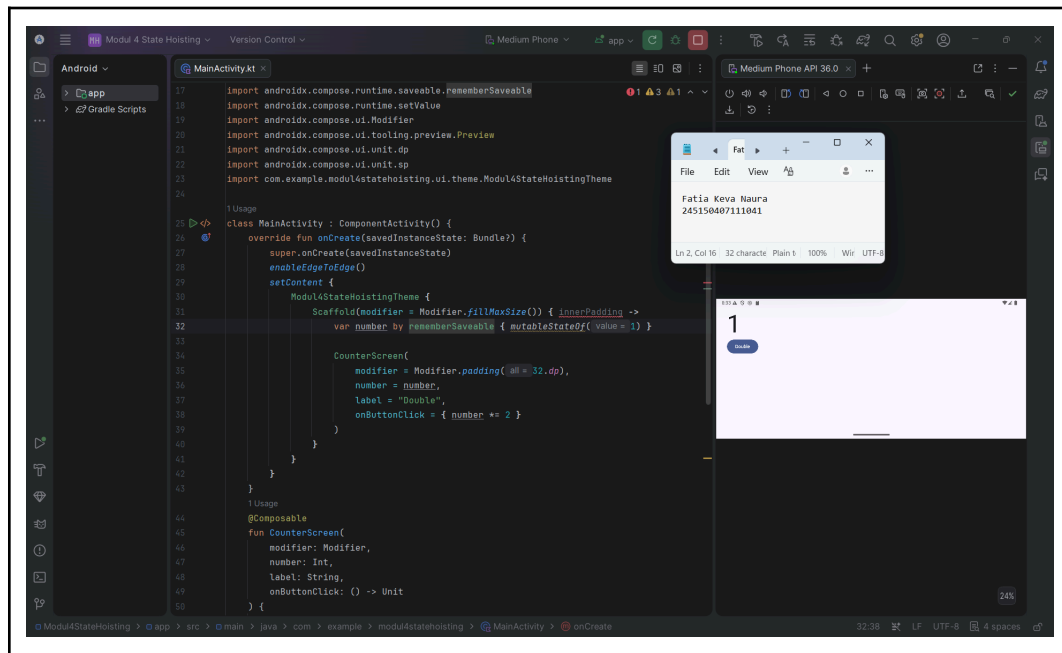
Sebelum tombol Double diklik:



Sesudah tombol Double diklik:



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

= Angka yang ditampilkan adalah 1. Hal ini terjadi karena saat posisi perangkat diubah dari portrait menjadi landscape, terjadi perubahan konfigurasi sehingga Activity dibuat kembali. Nilai yang menggunakan remember tidak dipertahankan dan kembali ke nilai awal, yaitu 1.

Aktivitas percobaan:

Ubah bagian kode program di dalam fungsi onCreate() seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

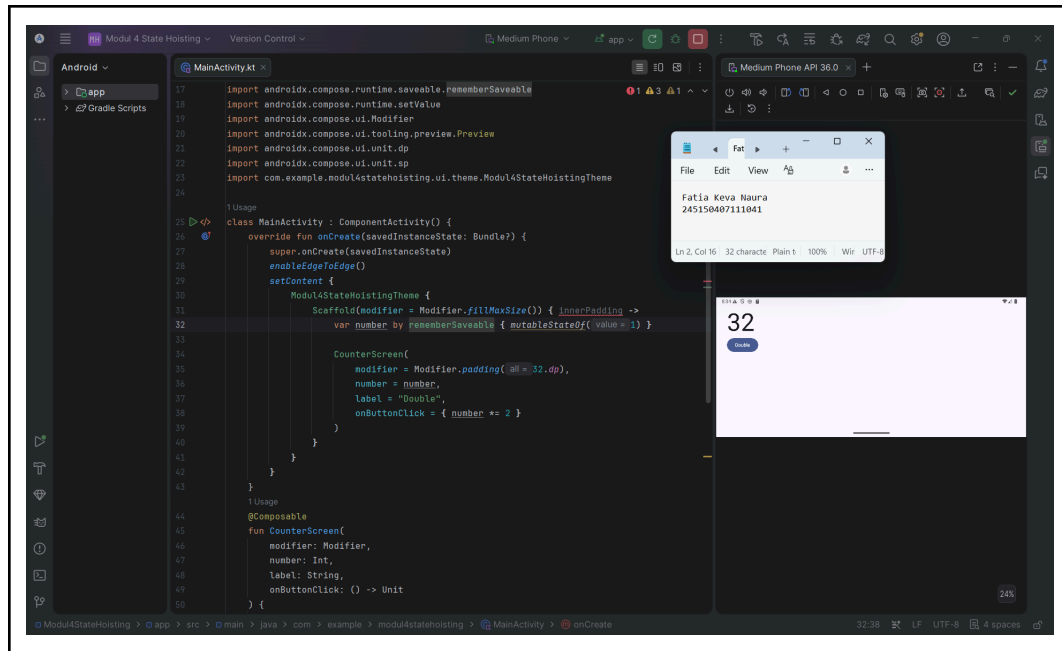
menjadi:

```
var number by rememberSaveable { mutableStateOf(1) }
```

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. **Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.**



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

= Angka yang ditampilkan adalah 32. Nilai tersebut tetap 32 karena menggunakan `rememberSaveable`, sehingga state dapat disimpan dan dipulihkan kembali ketika terjadi perubahan konfigurasi seperti rotasi layar.

c. Apa fungsi dari konstruk `rememberSavable` pada kode program Android yang membedakannya dari konstruk `remember`?

= rememberSaveable berfungsi untuk menyimpan dan memulihkan nilai state ketika terjadi perubahan konfigurasi, seperti saat layar diputar. Sedangkan remember tidak mempertahankan nilai state ketika Activity dibuat kembali akibat perubahan konfigurasi.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSavable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

= Contohnya pada aplikasi yang memiliki form pengisian data. Saat pengguna sedang mengisi form kemudian layar diputar, data yang sudah dimasukkan dapat tetap tersimpan dengan menggunakan rememberSaveable. Dengan begitu, pengguna tidak perlu mengisi ulang data yang sudah dimasukkan sebelumnya.

4.5.3 LaunchedEffect

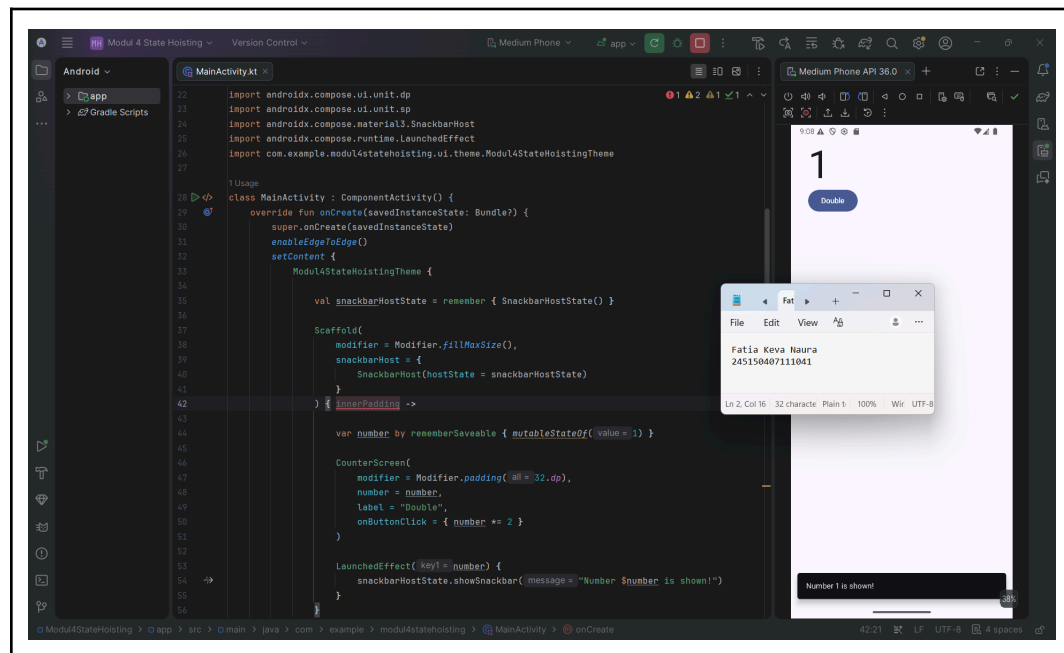
Pra-Percobaan 1

Aktivitas Percobaan

1. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

= Pada saat pertama kali program dijalankan, angka yang ditampilkan adalah 1 dan muncul pesan Snackbar "Number 1 is shown!".



- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

= Karena `LaunchedEffect(number)` langsung dijalankan ketika composable pertama kali masuk ke dalam layar. Nilai awal number adalah 1, sehingga Snackbar menampilkan pesan "Number 1 is shown!" walaupun pengguna belum mengklik tombol.

- c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

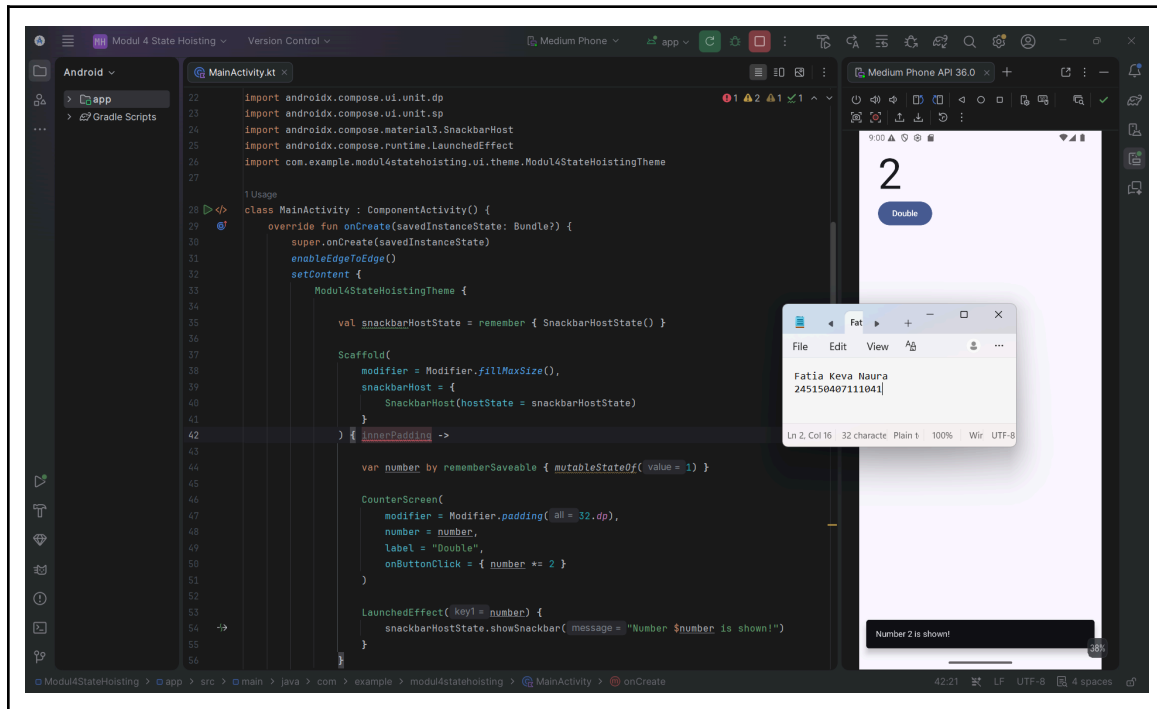
= Pesan Snackbar dipicu oleh perubahan atau nilai awal dari number pada `LaunchedEffect(number)`. Bagian kode yang menjalankannya adalah:

```

LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}

```

2. Klik button "Double" satu kali,



a. Apa yang terjadi?

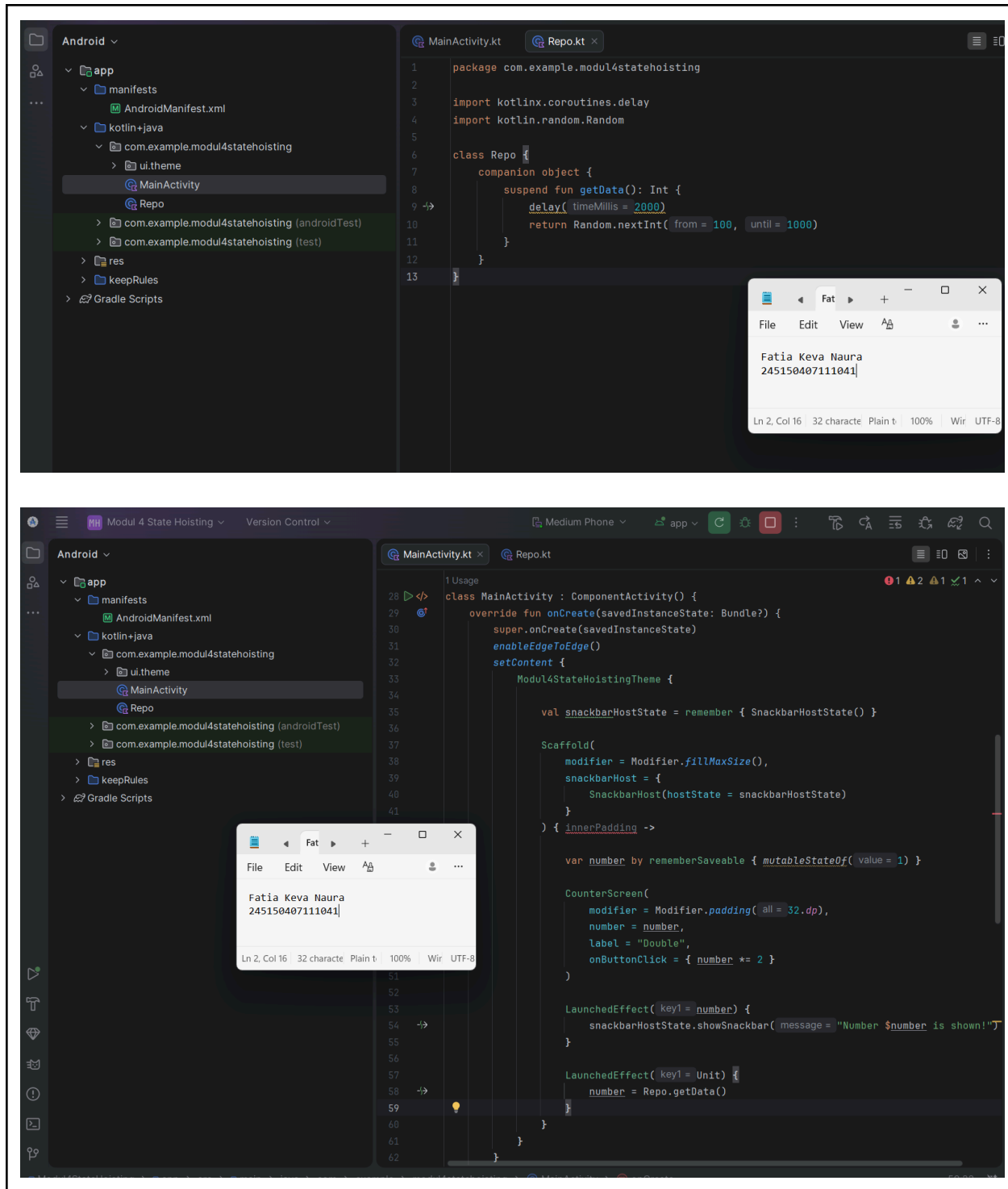
= Setelah tombol "Double" diklik satu kali, nilai angka berubah dari 1 menjadi 2. Setelah itu muncul Snackbar dengan pesan "Number 2 is shown!".

b. Mengapa hal tersebut terjadi? Jelaskan!

= Hal tersebut terjadi karena ketika tombol "Double" diklik, kode `number *= 2` dijalankan sehingga nilai number berubah dari 1 menjadi 2. Perubahan nilai tersebut kemudian memicu `LaunchedEffect(number)` untuk dijalankan kembali, sehingga Snackbar menampilkan pesan "Number 2 is shown!".

Pra-Percobaan 2

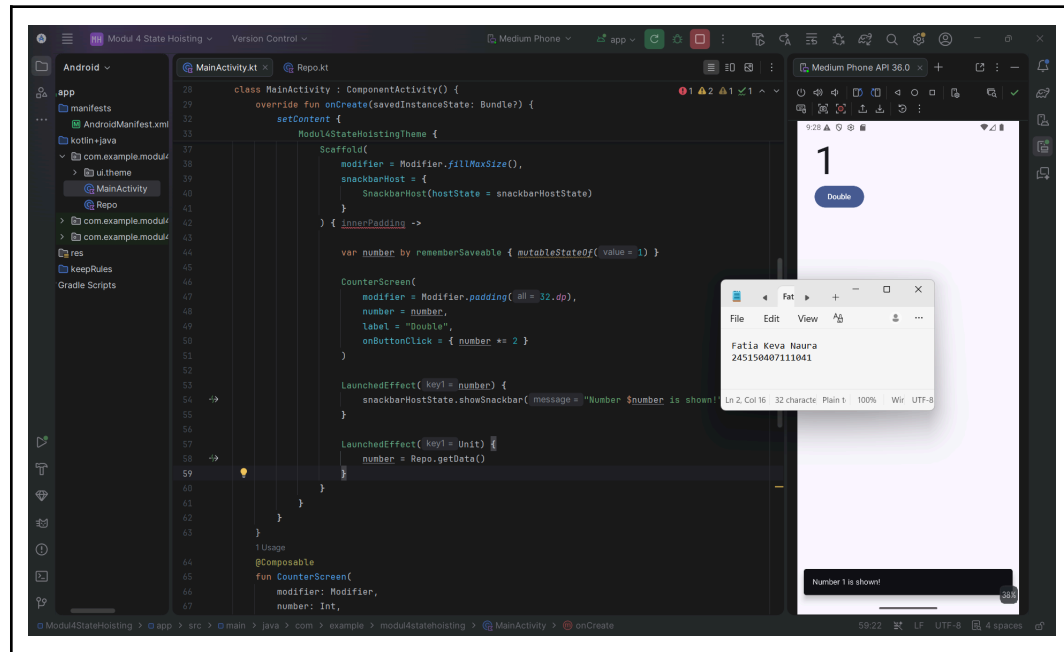
Aktivitas Percobaan



1. Jalankan program,

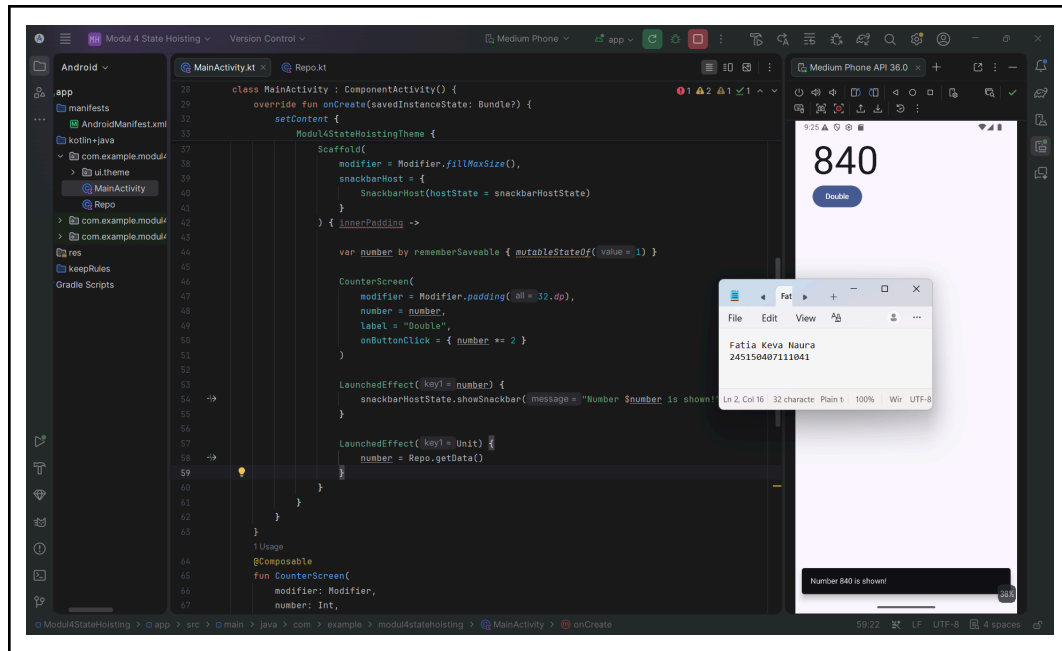
- a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

= Saat pertama kali program dijalankan, angka yang ditampilkan adalah 1 dan muncul Snackbar "Number 1 is shown!".



- b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

= Setelah menunggu sekitar 2 detik, angka 1 berubah menjadi angka acak. Pada percobaan saya, angka yang muncul adalah 840 dan muncul Snackbar "Number 840 is shown!".



c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

= Perubahan angka dipicu oleh `LaunchedEffect(Unit)` yang menjalankan `Repo.getData()`. Setelah mendapatkan angka baru, nilai `number` berubah. Perubahan tersebut kemudian memicu `LaunchedEffect(number)` sehingga `Snackbar` menampilkan angka yang baru.

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

= Perbedaannya terletak pada kondisi yang memicu `LaunchedEffect` untuk dijalankan kembali. `LaunchedEffect(state)` bergantung pada nilai `state`, sehingga ketika nilai `state` berubah, efek akan dijalankan kembali. Sedangkan `LaunchedEffect(Unit)` menggunakan `Unit` sebagai `key` sehingga efek dijalankan saat pertama kali `composable` masuk ke dalam `composition` dan tidak dijalankan kembali hanya karena nilai `state` lain berubah.

b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

= `LaunchedEffect(state)` dijalankan ketika composable pertama kali masuk ke dalam composition dan akan dijalankan kembali ketika nilai state yang menjadi key berubah. Pada percobaan, ketika nilai number berubah dari 1 menjadi angka acak, `LaunchedEffect(number)` dijalankan kembali dan menampilkan Snackbar dengan nilai terbaru.

Sedangkan `LaunchedEffect(Unit)` dijalankan ketika composable pertama kali masuk ke dalam composition. Pada percobaan kedua, `LaunchedEffect(Unit)` menjalankan `Repo.getData()` untuk mengambil angka secara acak setelah menunggu sekitar 2 detik.